

作业 2 Unix V6++内存分配，地址映射与内存保护

同济大学计算机系操作系统 国豪 2023 拔尖班 2023
姓名 毛星博 学号 2351520

一、内存分配 & 为进程建立地址映射关系



T0 时刻，用户区如图所示（注释：阴影部分已分配，除了 PA 正文段和可交换部分，还紧贴存放有其它进程的正文段和可交换部分）。PA 进程代码段 3 页，数据段 1 页，堆栈段 1 页；x_caddr=0x402000，p_addr=0x407000。PA 创建子进程 PB。回答以下问题

- 1、PB 进程， x_caddr = (0x402000)， p_addr = (0x410000)
- 2、画 PA 的相对表，PB 的相对表

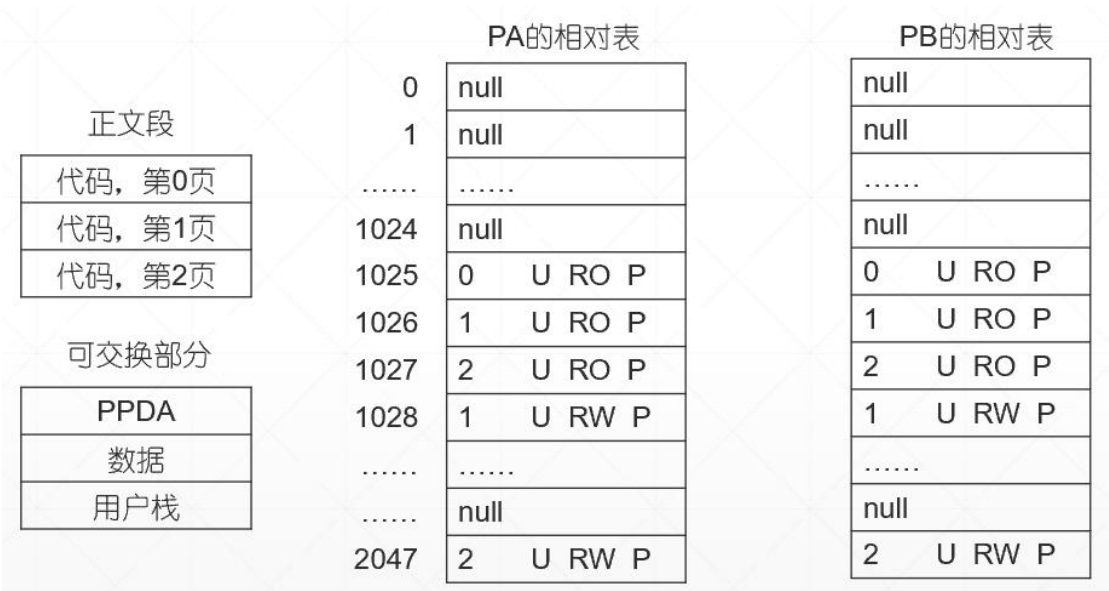
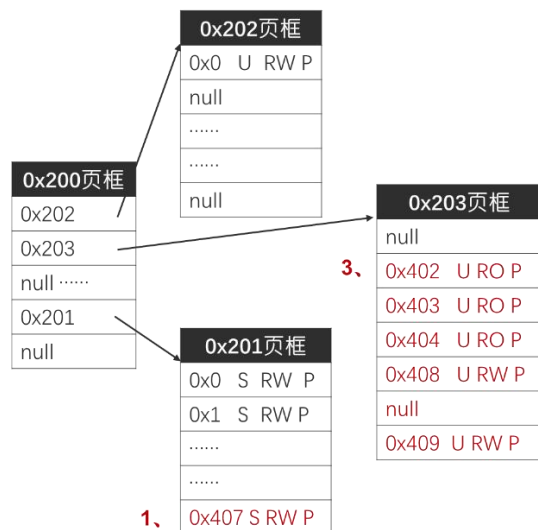


图 1

- 3、调度 PA，执行前为其建立地址映射关系



PA进程代码段3页，数据段1页，堆栈段1页；
x_caddr=0x402000, p_addr=0x407000

1、SwitchUStruct(selected);

p_addr >> 12, 填入核心页表1023#PTE的 base 字段, 建立PPDA区的地址映射关系。
#define SwitchUStruct(p) \ Machine::Instance().GetKernelPageTable().m_Entries[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \ = (p)->p_addr / PageManager::PAGE_SIZE; \ FlushPageDirectory();

2、User& newu = Kernel::Instance().GetUser();

newu 是装在物理页框0x407里, PA进程的user结构

3、newu.u_MemoryDescriptor.MapToPageTable();

以PA的相对表为模板, 写0x202,0x203#物理页框

图 2

4、调度 PB, 执行前为其建立地址映射关系

请阅读源代码, 画系统页表

5、进程切换是指 CPU 换现运行进程。上述第 3、4 小题展示进程切换时, 为新选中进程建立地址映射关系。请修改 MapToUserPageTable()函数, 尝试不使用相对表, 用 (1) 进程内存描述符中的其它字段 (2) x_caddr (3) p_addr 为新选中进程建立地址映射关系。写设计思路和伪代码。

二、地址映射 & 内存保护

例 1: 现运行进程, 1 页代码, 1 页堆栈。
x_caddr = 0x500000, p_addr = 0x525000。

(1) 填空

● 代码段第 1 条指令。

VA = (0x401000), PA = (0x500000)

● user 结构

VA = (0xC03ff000), PA = (0x525000)

注: VA 虚地址, PA 物理地址

(2) 请简述 i386 MMU 地址映射、内存保护过程

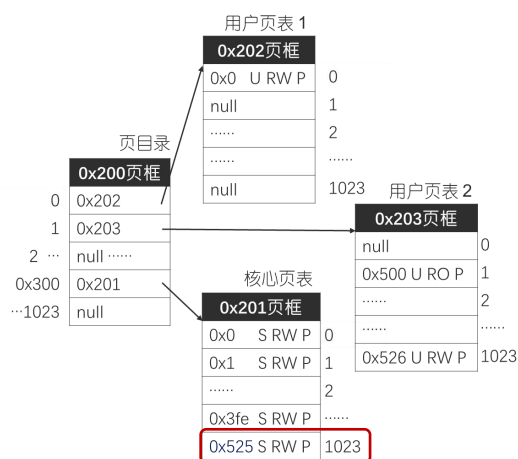
● CPU 用户态执行, 读操作, VA=0x401000。

解: 地址映射、内存保护, 过程如下:

第 1 步: 查二级页表

VA 展开成 2 进制数: 00000000010000000001000000000000, 分割

pagetable == 1, 查 cr3 指向的页目录, 得 PDE[1].base = 0x203, 映射需要的页表在 0x203# 物理页框;



page == 1, 查页表, 获得映射需要的 PTE, 用户页表 2 中的 PTE[1]。

第 2 步: 内存保护, 使用 PTE[1] 中的保护位 prot

P 为 1, 目标逻辑页存在

CPU 用户态运行 & U, 合法

内存读操作 & RO, 合法

第 3 步: 地址映射, 使用 PTE[1] 中的物理页框号 base

生成物理地址, 0x500000。

- mov %ebp, 0xC03ff000 (CPU 核心态运行时执行这条指令)

mov 指令写内存单元, 现运行进程的 user 结构的第 1 个字段。访问操作数时, CPU 使用的 VA=0xC03ff000。

查二级页表, 确定映射需要的 PTE: 核心页表、最后 1 个表项, 红框所示。

通过内存保护检测, 生成物理地址 0x525000。

- mov %ebp, 0xC03ff000 (CPU 用户态运行时执行这条指令)

CPU 用户态运行, 访问核心页面 (目标 PTE 有 S 标识)。非法, MMU 抛出 14#缺页异常, 杀死现运行进程, 阻止其破坏系统。

- 写 VA=0x401000 (CPU 用户态运行时执行这条指令)

目标 PTE 的属性是 RO, 只读。写只读逻辑页, 非法, MMU 抛出 14#缺页异常, 杀死现运行进程, 阻止其破坏系统。

- 读 VA=0xC01a8000 (CPU 用户态运行时执行这条指令)

目标 PTE 的属性是 S。应用程序访问核心逻辑页, 非法, MMU 抛出 14#缺页异常, 杀死现运行进程, 阻止其破坏系统。

- 读 VA=0x480000 (CPU 用户态运行时执行这条指令)

映射需要的 PTE 不存在, 非法~~~

- 读 VA=0x900000 (CPU 核心态运行时执行这条指令)

映射需要的 PTE 不存在, 非法~~~

[旁白, 指令的执行过程, call main1 为例]

call 指令读内存单元, 取 main1 函数第 1 条指令, 使用的 VA=0x401000。经地址映射后, 得 PA=0x500000。将其放地址总线, 1 个主存周期后, 数据总线上会出现 main1 函数的第 1 条指令, CPU 执行单元读数据总线获得该指令。译码、执行这条指令。

习题: PA 进程代码段 3 页, 数据段 1 页, 堆栈段 1 页; x_caddr=0x402000, p_addr=0x407000。

(1) 填空

- 代码段第 1 条指令。

VA = (), PA = ()

- user 结构

VA = (), PA = ()

(2) 请简述下述指令执行时, i386 MMU 地址映射、内存保护过程

- `call main1` (CPU 用户态运行时执行这条指令)
- `mov %ebp, 0xC03ff000` (CPU 核心态运行时执行这条指令)
- `mov %ebp, 0xC03ff000` (CPU 用户态运行时执行这条指令)
- 写 `VA=0x401000` (CPU 用户态运行时执行这条指令)
- 读 `VA=0xC01a8000` (CPU 用户态运行时执行这条指令)
- 读 `VA=0x480000` (CPU 用户态运行时执行这条指令)
- 读 `VA=0x900000` (CPU 核心态运行时执行这条指令)